

프로토콜 액션 → 소스 핸들러, 어떻게 정적으로 복원하는가

F2-A는 동적 디스패치가 지운 매핑을 **증거 기반 계산**으로 복원한다.
이 발표가 끝나면 알고리즘을 직접 재구성할 수 있어야 한다.

“이 액션을 처리하는 함수”를 정적으로 못 찾는다

질문 — RemoteStartTransaction은 어느 C 함수가 처리하는가?

디스패치 코드

```
void on_message(Frame *f) {  
    Entry e = g_table[ f->action ];    // 테이블 조회  
    e.fn(f->payload);                  // 간접 호출 — 대상이 코드에 없음  
}
```

시도 → 실패

grep remote → 함수 정의만, 매핑 없음

call graph → e.fn(...) 간선 없음

실행 → 정적·펌웨어 전체·하네스 없음 ✕

보안 질문은 액션 단위: “충전 프로파일 한계값이 검증되나?” → 먼저 어느 함수가 그 액션을 처리하는지 알아야 오염 분석이 시작된다.

간선은 제어 흐름이 아니라 데이터에 있다

질문 — 왜 콜그래프 순회가 실패하나?

CPG — 간접 호출 노드

```
CALL  e.fn(...)
├ receiver: e.fn          // 함수 포인터
└ CALL edge → ∅ 대상 없음 // 정적으로 미해결
```

연결이 실제로 있는 곳 (데이터)

```
g_table[ACTION_REMOTE_START] =
  (Entry){ .fn = remote_handler };
// 등록: 액션 ID ↔ 콜백이 여기서 묶임
```

전략 — 호출(제어 흐름)이 아니라 등록 구조 + 식별자 매칭으로 ACTION_REMOTE_START → remote_handler 간선을 복원한다.

인식은 근거만 만들고, 판정은 뒤에서 한 번만

질문 — 왜 추출기가 직접 결정하지 않나?



KB는 “한 액션의 모든 코드 표기”를 안다

질문 — 무엇과 무엇을 대조해서 매칭하나?

KB 프로파일 (프로토콜 진실)

```
RemoteStartTransaction:
  protocol_string: "RemoteStartTransaction"
  symbols:        [ACTION_REMOTE_START]
  numeric_ids:    [3]
  aliases:        [remote_start]
```

CPG/AST + 심볼 (코드 구조)

- **CPG/AST** — 초기화·대입·호출·switch 형태
- **심볼** — 참조가 내부 함수인지(외부 스텝 아님)
- **원시 식별자** — 코드 지점에 등장한 그대로

KB의 여러 표기(symbol·numeric·alias)와 코드의 원시 식별자를 대조 → **매치 강도**가 나온다

코드 표기가 정규 ID와 얼마나 정확히 맞는가

질문 — “매치 강도”는 구체적으로 어떻게 정해지나?

코드에 나타난 원시 식별자	정규 ID와의 관계	MATCHSTRENGTH	×배수
ACTION_REMOTE_START	KB symbol과 완전 일치	EXACT_IDENTIFIER	1.00
3	숫자 ID가 상수로 해석·일치	RESOLVED_VALUE	0.95
remote_start	정규화 이름 일치(별칭)	NORMALIZED_NAME	0.85
handle_remote_x	부분 문자열/토큰 겹침	HEURISTIC_SUBSTRING	0.70
– 없음 –	대응 없음	NONE	0.00

이 배수가 뒤의 모든 점수에 곱해진다. 정규화는 종류가 아니라 “대응의 정확도”를 수치화한다.

같은 의미, 다른 문법 — 그래서 추출기도 여럿

질문 — 인식기는 코드에서 무엇을 알아보나?

형태	코드	방출 증거
집합 초기화	{ ACTION, fn }	REGISTRATION_INIT
지정 초기화	{ .action = ACTION, .fn = fn }	REGISTRATION_INIT
인덱스 대입	handlers[ACTION] = fn	REGISTRATION_ASSIGN
상관 저장	t[k].action = ACTION; t[k].fn = fn	REGISTRATION_ASSIGN
등록 함수	register(ACTION, fn) → 콜리가 저장	REGISTRAR_CALL
디스패치	switch(action){ case ACTION: fn(); }	ENUM_CASE / STRING_DISPATCH

의미는 하나(**action** ↦ **handler**)지만 AST 형태가 달라, 단일 매치는 취약하다

인식기는 AST 형태로 id·콜백을 상관시킨다

질문 — 지정 초기화는 왜 따로 다뤄야 하나?

집합 초기화 — ID가 형제 노드

```
arrayInitializer "{ ACTION_REMOTE_START, remote_handler }"  
├─ ACTION_REMOTE_START      // id (형제)  
└─ METHOD_REF remote_handler // 콜백
```

지정 초기화 — ID가 형제 필드

```
arrayInitializer "{ .action=..., .fn=... }"  
├─ BLOCK └─ .action = ACTION_REMOTE_START // id 필드  
└─ BLOCK └─ .fn = remote_handler          // 콜백 필드
```

지정 멤버는 대입으로 감싸진다. 인식기는 “같은 struct 초기화에 속한 형제 필드”로 id↔콜백을 상관시킨다 — 순서 무관, 필드명 추측 없음.

모든 추출기의 출력 = 하나의 증거 레코드

질문 — 인식기는 정확히 무엇을 방출하나?

EVIDENCE (실제 객체)

```
Evidence {
  kind      : REGISTRATION_INIT
  callback  : remote_handler           // u.c:15
  action_id : { symbol: "ACTION_REMOTE_START",
                match : EXACT_IDENTIFIER }
  provenance : "site:reg:g_table"       // 그룹 키
  score      : 0.80                     // = 0.80 × 1.00
  records   : [
    DISPATCH_HANDLER_TABLE
    "{ ACTION_REMOTE_START, remote_handler }" u.c:12
    HANDLER_REF "remote_handler"           u.c:15
  ]
}
```

필드가 하는 일

kind — 어떤 방식이 만들었나 (사전확률)
action_id.match — 매치 강도 (배수)
provenance — 중복 제거·그룹화 키
score — prior × 배수
records — 파일·라인 추적 = 감사 가능

여러 추출기가 동시에 방출 → 같은 바인딩에 대한 독립 관찰들이 계산 계층으로 넘어간다

한 증거의 점수 = 사전확률 × 매치 배수

질문 — 종류와 매치 강도가 점수로 어떻게 합쳐지나?

`score = Prior(kind) × MatchMult(strength)`

·

식별자 상충 시: `Prior × min(Mult, 0.70) × 0.5`

사전확률 PRIOR(KIND)	
STRING_DISPATCH	0.90
ENUM_CASE	0.85
REGISTRATION_INIT / ASSIGN	0.80
REGISTRAR_CALL · NAME_MATCH	0.70

계산 예시				
증거	PRIOR	×배수	SCORE	
REG_INIT · EXACT	0.80	1.00	0.80	
ENUM_CASE · NORMALIZED	0.85	0.85	0.72	
REG_INIT · SUBSTRING	0.80	0.70	0.56	
NAME_MATCH · SUBSTRING	0.70	0.70	0.49	

디스패치(0.85)가 등록(0.80)보다 사전확률이 높다 — 종류가 아니라 점수가 승부를 가른다. 아직 후보 병합 없음.

중복은 MAX로 접고, 독립 근거는 noisy-OR로

질문 — 여러 증거 → 후보 신뢰도 하나, 어떻게?

의사코드 (후보당)

```
for g in provenance_groups(evidence):
    group[g] = max(score(e) for e in g) # 같은 사이트=한 번
conf = 1 - ∏(1 - group[g]) # noisy-OR: 독립 확증
if 강한매치_없음: conf = min(conf, 0.85) # weak-only cap
conf = min(conf, 0.99) # global cap
```

수치 예시

MAX 같은 등록 두 관찰 0.80, 0.80
→ 그룹 **0.80** (합 아님 — 증인 하나)
noisy-OR 독립 그룹 0.80, 0.49
→ $1 - (0.20 \times 0.51) = \mathbf{0.898}$
두 약한 독립 근거가 각자보다 커짐

왜 noisy-OR? 독립 확증은 신뢰를 올리되 **확실성엔 못 미친다**(global cap 0.99). 왜 MAX? 같은 사이트의 두 레코드는 두 증인이 아니다.

임계값 + 마진 — 구조 게이트는 없다

질문 — **확정/모호/미해결은 정확히 무슨 조건인가?**

의사코드

```
후보 = sort_desc(by conf)
top = 후보[0]
if top.conf < 0.50: # Acceptance Floor
    return UNRESOLVED(LOW_CONFIDENCE)
if top.conf - 후보[1].conf < 0.15: # Ambiguity Margin
    return AMBIGUOUS(경쟁 보존)
return RESOLVED(top)
```

결정 표

1위	2위	결과
0.80	0.40	확정
0.72	0.56	확정 (마진 0.16)
0.66	0.56	모호 (마진 0.10)
0.42	—	미해결 (<0.50)

잘못된 확정은 후속 전체를 오염시킨다 → 근거가 갈리면 **의도적으로 보류**. 강한 매치의 디스패치는 **단독으로도 확정**된다(구조 종류 무관).

등록 vs switch — 전 과정을 수치로 따라가기

질문 — 같은 액션에 후보가 둘이면 누가 이기나?

```
static Reg t[] = { { ACTION_DATA_TRANSFER, foo } }; // 등록
switch (f->action){ case ACTION_DATA_TRANSFER: bar(); } // 디스패치
```

후보	KIND	매치	PRIOR×배수	CONF
bar	ENUM_CASE	NORMALIZED	0.85 × 0.85	0.72
foo	REGISTRATION_INIT	SUBSTRING	0.80 × 0.70	0.56

1위 0.72 ≥ 0.50 · 마진 0.72-0.56 = 0.16 ≥ 0.15 → **RESOLVED** → bar

반사실: switch를 지우면 증거는 foo 하나 → 0.56 ≥ 0.50, 경쟁 없음 → **RESOLVED** → foo. 승부는 종류가 아니라 매치 강도가 갈랐다.

미해결엔 “왜 막혔는지”가 코드로 붙는다

질문 — 못 풀었을 때 무엇을, 왜 보고하나?

지원됨 — 슬롯이 정적으로 보임

```
t[k].action = ACTION; // id 저장
t[k].fn     = fn;     // 같은 슬롯 k
```

미지원 — 런타임 탐색 후 저장

```
for (i..) if (t[i].action == a) // 비교(솔어)
{ t[i].fn = fn; return; } // 심볼릭 i
```

미해결 사유 (구조화)

증거 없음	NO_EVIDENCE
임계값 미달	LOW_CONFIDENCE
저장 미도달	REGISTRAR_STORE_NOT_REACHED
탐색-후-저장	REGISTRAR_SEARCH_THEN_WRITE
외부 정의	EXTERNAL_DEFINITION
동적 액션 ID	DYNAMIC_ACTION_ID

id가 **저장이 아니라 비교**만 되고 슬롯이 심볼릭 i라, 정적 쌍-저장 추출기가 닿지 못함 → 버그가 아니라 **명명된 범위 밖**

미해결도 근거와 함께 실려 나가는 정식 결과

질문 — 후속 단계는 무엇을 받나?

```
"RemoteStartTransaction": {
  "status": "RESOLVED",
  "chosen": { "function": "remote_handler", "file": "u.c", "line": 15 },
  "candidates": [ { "function": "remote_handler", "confidence": 0.80,
                    "evidence_kinds": ["REGISTRATION_INIT"], "evidence": [ ... records ... ] } ],
  "conflict": null, "unresolved": null
},
"UpdateFirmware": {
  "status": "UNRESOLVED",
  "unresolved": { "reason": "NO_EVIDENCE", "attempted": ["registration_ast", "name_match"] }
}
```

handler_resolutions(권위·액션당 하나) + **handler_maps**(확정 전용·하위 호환) + **limitations**. 모든 후보가 증거·추적으로 연결된다.

확정 핸들러 = 오염 분석의 출발 함수

질문 — 이 매핑이 실제로 무엇을 열어주나?

F6가 핸들러에서 확장

```
int remote_handler(Payload *p) { // 소스 루트
    int lim = p->chargingLimit; // 필드 바인딩
    set_current_limit(lim); // 싱크 - 검증 없음?
}
```

확장 경로

확정 핸들러

- 필드 바인딩 (필드→변수)
- 소스 식별 (오염 루트)
- 싱크 도달성
- 기대 검사 매칭 → **F6 결과**

핸들러가 없으면 소스 루트도 없다 → 그 액션엔 결과가 없다. 그래서 **확신에 찬 오답보다 정직한 보류가 낫다.**

구조로 결정 가능하면 지원, 값 추론이면 보류

질문 — 경계는 정확히 어디인가?

지원 (AST + 심볼 + 알은 호출)

집합·지정·인덱스·상관 등록
등록 함수 추적 (저장 도달)
위임 등록 (깊이 ≤ 2)
enum/switch 디스패치 · 이름 풀백
모호성 보류 · 구조화 진단

미지원 → 필요 역량

탐색-후-저장	루프+술어 추론
런타임 슬롯 선택	심볼릭 인덱스
별칭→슬롯	points-to 분석
프로시저 간 슬롯	프로시저 간 값 흐름
동적 액션 ID	DFG 에스컬레이션

미지원도 탐지·명명 후 보류 — 정밀·설명 가능한 코어 + 정직한 공백

픽스처마다 기대 결과를 못박아 회귀를 막는다

질문 — 이 동작이 맞다는 걸 어떻게 아나?

픽스처 (코드 형태)	기대 상태	KIND / 사유
{ ACTION, fn }	RESOLVED	REGISTRATION_INIT
{ .action=ACTION, .fn=fn }	RESOLVED	REGISTRATION_INIT
등록 vs switch	RESOLVED	bar · ENUM_CASE
두 등록 경쟁	AMBIGUOUS	margin < 0.15
탐색-후-저장	UNRESOLVED	REGISTRAR_SEARCH_THEN_WRITE
저장 없는 등록 함수	UNRESOLVED	REGISTRAR_STORE_NOT_REACHED

역량 테스트(된다) · 회귀 테스트(드리프트 방지) · 미지원 테스트(**올바른 거절**을 단언) — “크래시 없음”이 아니라 정확한 결과를 못박음

알고리즘 한 장 요약 — 이제 재구성할 수 있다

핵심 — 인식과 판정의 분리가 이 설계의 기여다.

```
# 액션 a 에 대해
ev = [] extractor(a) for extractor in EXTRACTORS # 인식 → 증거
ev = dedupe(ev)
for e in ev: e.score = Prior(e.kind) × MatchMult(e.match) # 점수
for cand in group_by_callback(ev): # 결합
    g = { grp: max(score) per provenance group }
    conf = 1 - ∏(1 - g[grp]) # noisy-OR
    if no strong match: conf = min(conf, 0.85)
top, second = rank_desc(cands) # 판정
return UNRESOLVED if top<0.50
    else AMBIGUOUS if top-second<0.15
    else RESOLVED(top)
```

보수성은 임계값·충돌 처리·출처 인지 결합·명시적 보류에서 온다 — 구조 게이트가 아니라. 감사 가능성은 모든 해석이 증거 레코드로 추적되기 때문.
확장성은 새 추출기가 계산·판정을 건드리지 않기 때문.

파라미터 · 메커니즘 (질의응답 예비)

상수	값	역할
Acceptance Floor	0.50	이하면 UNRESOLVED
Ambiguity Margin	0.15	1·2위 차 이하면 AMBIGUOUS
weak-only cap	0.85	정확 식별자 매치 없을 때 상한
global cap	0.99	확실성 방지 상한
conflict penalty	×0.5	식별자 상충(배수 cap 0.70)
registrar depth	2	등록 함수 추적 최대 홉

A2 출처 그룹화(그룹 내 MAX·그룹 간 noisy-OR, 이중 계산 방지) · A3 지정 vs 집합 AST lowering 차이